Lab 2 – FreeRTOS Setup and Basic Motor Control (Open-Loop)

Student Names: Krisha Veera, Stuti Pandya, Lana Othman, Sahil Shukla, Salim Aissaoui

Group #12

SEG4145 Real Time Systems Design

Professor: Mohamed Ali Ibrahim

Presented to: Pankaj Rathi

School of Electrical Engineering and Computer Science École de science informatique et de génie électrique

Faculty of Engineering | Faculté de génie University of Ottawa | Université d'Ottawa

Submission Date: February 14, 2026

Introduction

The objective of this laboratory was to design and implement a real-time embedded system using FreeRTOS to perform open-loop DC motor control. The system integrates multiple peripherals including PWM generation via TIM1 (PE9), ADC sampling on PC0, GPIO control for direction (PB10/PB11) and standby (PE15), UART logging, and non-blocking buzzer feedback.

The system follows a task-based architecture using CMSIS-RTOS2 on top of FreeRTOS. The potentiometer value is sampled using ADC1 to adjust speed, while motor state transitions (STOP → CW → CCW) are triggered by a push button interrupt and synchronized with the tasks using RTOS semaphores.

This lab establishes a scalable and deterministic real-time architecture that provides the foundation for future closed-loop control laboratories.

Equipment

Hardware

- STM32L552ZE Microcontroller Board (Cortex-M33)
- DC Motor & L298N/Equivalent Motor Driver
  - PWM Speed Control: PE9 (TIM1_CH1)
  - Direction Control: PB10 (AI1), PB11 (AI2)
  - Standby (STB): PE15
- Potentiometer: Connected to PC0 (ADC1)
- User Feedback:
  - On-board LEDs: Red (Stop), Green (CW), Blue (CCW)
  - Buzzer: Connected to PA0 (TIM5 PWM output)
  - User Push Button: Connected to PC13 (External Interrupt)

Software

- **STM32CubeIDE:** Integrated Development Environment for C/C++ and Configuration (iOC).
- **FreeRTOS (CMSIS-RTOS2 API):** Real-time kernel for task scheduling and synchronization.
- **STM32CubeL5 HAL Drivers:** Hardware Abstraction Layer for peripheral control.
- **Serial Terminal (e.g., PuTTY):** For monitoring UART debug logs.

Design Process

3.1 Overall Architecture

The system is designed using a preemptive multitasking architecture managed by FreeRTOS (CMSIS-RTOS2). To ensure determinism and safety, the system is decomposed into two primary functional tasks and supporting service tasks:

- **MotorTask (Priority: AboveNormal / High):** This is the highest priority functional task. It is responsible for hardware-timed motor control, updating PWM registers (TIM1), and managing direction GPIOs. High priority ensures that motor control logic is never delayed by user-interface processing.
- **UITask (Priority: Low):** This task handles human-interface elements, including sampling the potentiometer via ADC1 and processing speed change requests. Since humans perceive delays in milliseconds, this task runs at a lower priority to yield CPU time to the MotorTask.
- **Service Tasks:**
  - LoggerTask: Handles asynchronous UART transmission of system telemetry.
  - SysStatusTask: Monitors system health via periodic heartbeat messages.
  - Button Handling: Managed via an External Interrupt (EXTI) and a Binary Semaphore, ensuring an immediate, non-blocking response to user input.

3.2 Motor State Design and Shared Control Variables

To structure motor behavior in a clear and scalable way, a motor state machine was implemented using an enumerated data type. The motor operating modes were defined as STOP, CW (clockwise), and CCW (counter-clockwise). This formal representation prevents ambiguous logic and ensures that only valid motor states can be requested.

```
typedef enum {
    MOTOR_STOP = 0,
    MOTOR_CW,
    MOTOR_CCW
} MotorState_t;

volatile MotorState_t uiMotorState = MOTOR_STOP;
volatile uint32_t uiPwmDuty = 0;
```

Two shared control variables were introduced: `uiMotorState` and `uiPwmDuty`. The variable `uiMotorState` stores the requested motor mode, while `uiPwmDuty` stores the requested PWM duty cycle (0–65535, mapped from the 12-bit ADC range). These variables are declared as `volatile` to ensure memory consistency across tasks. To comply with the requirement for safe communication mechanisms, access to these variables is synchronized. Since the variables are simple scalar types, the MotorTask reads them at the start of its 15 ms period to ensure a local, stable copy of the command is used for the duration of the control cycle, preventing race conditions.

This design separates decision-making from actuation:

- The UI layer (UITask) determines what the motor should do based on user input.
- The Actuation layer (MotorTask) applies those requests to the hardware.

3.3 Motor Task – Deterministic Actuation Layer

The MotorTask was implemented as the primary hardware control task. It is responsible for applying all motor-related outputs, including PWM duty cycle via TIM1 (PE9), direction signals via PB10 and PB11, standby control via PE15, and LED state indication.

The task executes periodically using `vTaskDelayUntil()` with a fixed period of 15 ms. This scheduling method ensures consistent execution intervals and minimizes timing jitter, which is essential for deterministic motor control.

Initial State: During initialization, the MotorTask explicitly forces the system into the STOP state. The PWM is set to zero, the standby pin (PE15) is driven LOW, both direction pins are reset, and the Red LED is turned ON. This guarantees the motor remains stationary upon reset, satisfying the mandatory initial condition requirement.

```
void StartMotorTask(void *argument)
{
    TickType_t xLastWakeTime = xTaskGetTickCount();
    const TickType_t xPeriod = pdMS_TO_TICKS(15);

    HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);

    uiMotorState = MOTOR_STOP;
    uiPwmDuty = 0;
    __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, 0);
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_15, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_10|GPIO_PIN_11, GPIO_PIN_RESET);
    BSP_LED_On(LED_RED);
    BSP_LED_Off(LED_GREEN);
    BSP_LED_Off(LED_BLUE);

    for (;;)
    {
        switch (uiMotorState)
        {
            case MOTOR_STOP:
                __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, 0);
                HAL_GPIO_WritePin(GPIOE, GPIO_PIN_15, GPIO_PIN_RESET);
                HAL_GPIO_WritePin(GPIOB, GPIO_PIN_10|GPIO_PIN_11, GPIO_PIN_RESET);
                BSP_LED_On(LED_RED);
                BSP_LED_Off(LED_GREEN);
                BSP_LED_Off(LED_BLUE);
                break;
            case MOTOR_CW:
                __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, uiPwmDuty);
                HAL_GPIO_WritePin(GPIOE, GPIO_PIN_15, GPIO_PIN_SET);
                HAL_GPIO_WritePin(GPIOB, GPIO_PIN_10, GPIO_PIN_RESET);
                HAL_GPIO_WritePin(GPIOB, GPIO_PIN_11, GPIO_PIN_SET);
                BSP_LED_Off(LED_RED);
                BSP_LED_On(LED_GREEN);
                BSP_LED_Off(LED_BLUE);
                break;
            case MOTOR_CCW:
                __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, uiPwmDuty);
                HAL_GPIO_WritePin(GPIOE, GPIO_PIN_15, GPIO_PIN_SET);
                HAL_GPIO_WritePin(GPIOB, GPIO_PIN_10, GPIO_PIN_SET);
                HAL_GPIO_WritePin(GPIOB, GPIO_PIN_11, GPIO_PIN_RESET);
                BSP_LED_Off(LED_RED);
                BSP_LED_Off(LED_GREEN);
                BSP_LED_On(LED_BLUE);
                break;
        }
        vTaskDelayUntil(&xLastWakeTime, xPeriod);
    }
}
```

Within its periodic loop, the MotorTask evaluates the current state and applies the following logic:

- **STOP Mode:** PWM = 0; PE15 (STB) = LOW; PB10 = LOW, PB11 = LOW; Red LED ON.
- **CW Mode:** PWM = uiPwmDuty; PE15 (STB) = HIGH; PB10 = LOW, PB11 = HIGH; Green LED ON.
- **CCW Mode:** PWM = uiPwmDuty; PE15 (STB) = HIGH; PB10 = HIGH, PB11 = LOW; Blue LED ON.

To satisfy the "Only one LED active" requirement, the task explicitly clears the inactive LED GPIO pins whenever a state transition occurs. By centralizing all hardware writes within this high-priority task, the system maintains a predictable and safe operating environment.

3.4 PWM Configuration (TIM1)

Motor speed control was implemented using TIM1 Channel 1 (PE9) configured in PWM Generation CH1 mode. The timer was configured with Prescaler = 0 and Period = 65535, resulting in a PWM frequency of approximately 1.68 kHz (110 MHz / 65536), which is suitable for DC motor control. The PWM output is initiated once during task initialization and updated dynamically by writing to the Capture Compare Register (CCR1).

The 12-bit ADC value (0–4095) is mapped to the PWM duty cycle range (0–65535). This ensures smooth and proportional speed control across the full rotation of the potentiometer. Following safety requirements, speed updates are applied only when the motor is in CW or CCW mode. In STOP mode, the PWM compare value is

forced to 0, and the Standby pin (PE15) is driven LOW to ensure the motor remains completely stationary regardless of the potentiometer position.

3.5 UI Task – Speed Acquisition and Control Request

The UITask (Priority: Low) is responsible for acquiring user input and converting it into a speed request. It runs periodically with a 30 ms execution interval, providing responsive control while yielding CPU time to higher-priority tasks.

To avoid blocking the processor, the ADC is configured in Interrupt Mode. The task initiates a conversion and then waits on a Binary Semaphore which is released by the `HAL_ADC_ConvCpltCallback`. Once unblocked, the task reads the 12-bit value from PC0.

```
void StartUITask(void *argument)
{
    TickType_t xLastWakeTime = xTaskGetTickCount();
    const TickType_t xPeriod = pdMS_TO_TICKS(30);
    uint32_t adcValue = 0;

    for (;;)
    {
        HAL_ADC_Start_IT(&hadc1);

        if (osSemaphoreAcquire(adcSemaphoreHandle, pdMS_TO_TICKS(5)) == osOK)
        {
            adcValue = HAL_ADC_GetValue(&hadc1);

            if (uiMotorState != MOTOR_STOP)
            {
                uiPwmDuty = (adcValue * htim1.Init.Period) / 4095;
            }
        }

        vTaskDelayUntil(&xLastWakeTime, xPeriod);
    }
}
```

The task then scales this value and assigns it to the protected shared variable `uiPwmDuty`. By decoupling the ADC sampling from the MotorTask, we ensure that slow analog-to-digital conversions do not introduce jitter into the high-priority motor control loop.

3.6 Button Task – Event-Driven State Transitions

The User Push Button (PC13) was configured using EXTI interrupt mode on the falling edge. To maintain a clean RTOS architecture, the Interrupt Service Routine (ISR) does not process logic; instead, it simply releases a Binary Semaphore to signal that an event has occurred.

```
void BSP_PB_Callback(Button_TypeDef Button)
{
    if (Button == BUTTON_USER)
    {
        osSemaphoreRelease(buttonSemaphoreHandle);
    }
}
```

The state transition logic follows the mandatory sequence: **STOP → CW → CCW → STOP**

```
void startButtonTask(void *argument)
{
    for(;;)
    {
        osSemaphoreAcquire(buttonSemaphoreHandle, osWaitForever);

        switch(uiMotorState)
        {
            case MOTOR_STOP:  uiMotorState = MOTOR_CW;   break;
            case MOTOR_CW:    uiMotorState = MOTOR_CCW;  break;
            case MOTOR_CCW:
            default:          uiMotorState = MOTOR_STOP;  break;
        }

        // Activate buzzer (non-blocking)
        __HAL_TIM_SET_COMPARE(&htim5, TIM_CHANNEL_1, 250);
        HAL_TIM_PWM_Start(&htim5, TIM_CHANNEL_1);
        osTimerStart(buzzerTimerHandle, 100);

        // Log state change
        osMessageQueuePut(logQueueHandle, &logMsg, 0, 0);
    }
}
```

When the signal is received, the motor state is updated in the shared variable `uiMotorState`. This event-driven approach is superior to polling because it allows the task to remain in a "Blocked" state (consuming zero CPU cycles) until the exact moment the user interacts with the hardware.

3.7 Buzzer Implementation – Non-Blocking Design

The buzzer feedback was implemented on PA0 using TIM5 to generate a 2 kHz PWM signal. TIM5 was configured with Prescaler = 109 and Period = 499, yielding a frequency of 110 MHz / 110 / 500 = 2 kHz. To meet the strict "No Blocking" requirement (Section 9 of the lab manual), the 100 ms beep duration is managed by a FreeRTOS Software Timer.

When a button press is detected:

1. `HAL_TIM_PWM_Start()` is called to begin the 2 kHz tone.
2. A One-Shot Software Timer is started with a 100 ms period.
3. The task continues execution immediately (non-blocking).
4. When the timer expires, the Callback Function executes `HAL_TIM_PWM_Stop()`.

```
void BuzzerTimerCallback(void *argument)
{
    HAL_TIM_PWM_Stop(&htim5, TIM_CHANNEL_1);
}
```

This design ensures that neither the UI Task nor the Motor Task is ever "stalled" by a `HAL_Delay()`, preserving the real-time responsiveness of the system.

3.8 Inter-Task Communication and Synchronization

The system utilizes several RTOS primitives to ensure a deterministic and "race-condition free" environment:

- **Binary Semaphores:** Used for ISR-to-Task synchronization (Button and ADC complete).
- **Message Queues:** Used for the LoggerTask, allowing other tasks to send UART strings without waiting for the slow serial hardware to finish transmitting.
- **Software Timers:** Used for non-blocking timing of the buzzer feedback.
- **Volatile Variables:** Used for `uiMotorState` and `uiPwmDuty`, ensuring the compiler does not optimize out memory reads across different task contexts.

3.9 Task Priority Logic

To ensure system stability, the MotorTask was assigned a higher priority than the UITask. This prevents the motor control loop from being interrupted by user-interface processing (like ADC sampling or UART logging). This hierarchy ensures deterministic hardware updates, which is critical for real-time safety, while the lower-priority UITask utilizes remaining CPU cycles for human-centric interactions.

Task Name	Priority Level	Priority Value	Stack Size	Role
MotorTask	Highest	osPriorityAboveNormal	512 bytes	Motor hardware control
LoggerTask	High	osPriorityAboveNormal	512 bytes	UART communication
DefaultTask	Normal	osPriorityNormal	512 bytes	System idle

Task Name	Priority Level	Priority Value	Stack Size	Role
UITask	Low	osPriorityLow	512 bytes	ADC speed control
ButtonTask	Low	osPriorityLow	512 bytes	State transitions
SysStatusTask	Low	osPriorityLow	512 bytes	Heartbeat logging

Results and Verification

The system was tested to verify compliance with all functional and architectural requirements specified in the laboratory instructions. Testing included reset behavior, task execution, motor state transitions, speed control, LED behavior, and buzzer feedback.

4.1 FreeRTOS Scheduler and Task Execution

Upon system startup, the FreeRTOS scheduler was successfully initialized. Multiple tasks were observed executing concurrently with the following verified timing:

- **MotorTask:** Executed every 15 ms using `vTaskDelayUntil()`, providing deterministic control.
- **UITask:** Executed every 30 ms, providing responsive user input sampling.
- **Button/Logger Tasks:** Remained in the Blocked state when idle, confirming efficient CPU utilization.

UART logs confirmed that tasks were running concurrently without any "stalls." The system heartbeat message was transmitted over UART every second, verifying that the scheduler remained active and stable.

4.2 Initial State Verification

Immediately after a hardware reset, the system correctly entered the mandatory initial state:

- **Motor State:** STOP
- **Hardware State:** PE15 (STB) driven LOW; PB10/PB11 driven LOW.
- **PWM Duty Cycle:** 0% (PE9).
- **LED Indication:** Red LED ON; Green and Blue LEDs OFF.
- **Motor Behavior:** Stationary.

This confirms compliance with the safety requirement that the motor must not rotate unintentionally upon system power-up.

4.3 Motor Direction Verification

Motor direction was tested by cycling through states using the user button. The observed behavior matched the requirements perfectly:

1. **First Press (CW):** Green LED ON; PB10=LOW, PB11=HIGH.
2. **Second Press (CCW):** Blue LED ON; PB10=HIGH, PB11=LOW.
3. **Third Press (STOP):** Red LED ON; PB10=LOW, PB11=LOW.

Verification of "Only one LED active": During transitions, the previous LED was successfully extinguished before the new state LED was illuminated, ensuring clear visual feedback.

4.4 Speed Control (Open-Loop Verification)

The potentiometer (on PC0) was rotated to verify open-loop speed control. The 12-bit ADC value was successfully mapped to the TIM1 compare register.

- **Responsiveness:** Increasing the potentiometer position resulted in a smooth, proportional increase in motor RPM.
- **State Constraint:** PWM remained at zero when the motor was in the STOP state, regardless of potentiometer position, satisfying the safety logic.

4.5 Buzzer Feedback Verification

Each button press triggered an audible 2 kHz beep. Crucially, the use of a FreeRTOS Software Timer meant that the 100 ms beep did not block the task execution.

- The system remained responsive to further inputs during the beep.
- No jitter was observed in the MotorTask timing during buzzer activation.

4.6 Logging and System Stability

UART logging provided real-time telemetry of state changes (e.g., "[EVENT] Button: CW") and system heartbeat messages. The system operated continuously during extended testing without crashes or unintended resets, proving the stability of the task architecture and the effectiveness of using semaphores for synchronization.

Discussion

This laboratory demonstrated the implementation of a real-time embedded control system using FreeRTOS. The primary objective was to design a deterministic and modular architecture that follows real-time system principles, ensuring that time-critical motor control is never compromised by secondary tasks.

Architectural Separation

A key design decision was separating decision-making from hardware actuation. The UITask and Button ISR determine the requested motor state and speed, while the MotorTask applies these requests periodically. This separation ensures that hardware control remains centralized. By centralizing all writes to TIM1 (PE9) and GPIO (PB10/11) within one high-priority task, we eliminate the risk of conflicting hardware commands.

Task Prioritization and Determinism

The MotorTask was assigned a higher priority because it manages physical actuation. Deterministic updates are essential for safe motor operation; using `vTaskDelayUntil()` ensures a fixed-period execution of 15 ms and minimizes timing jitter. In contrast, the UITask operates at a lower priority. While sampling the potentiometer on PC0 is important, a human user cannot perceive millisecond-level jitter in speed updates, making it a non-critical background operation.

Efficient Synchronization

To maximize CPU efficiency, the system avoids "busy-waiting" (polling). Event-driven synchronization was implemented using Binary Semaphores for button presses and ADC completion. This allows tasks to remain in a Blocked state, consuming zero CPU cycles until the hardware is ready. Furthermore, the buzzer implementation using a FreeRTOS Software Timer is a critical design choice. It allows for a 100 ms pulse without using `HAL_Delay()`, thus satisfying the requirement that no task should block the scheduler.

Future Improvements

The current system operates in open-loop mode, where the PWM duty cycle is set directly by the ADC. While effective for this lab, open-loop control cannot compensate for load variations or battery voltage drops. To improve the design in future laboratories, a closed-loop feedback mechanism (such as a PID controller using an encoder) could be integrated into the MotorTask to maintain a constant RPM regardless of external torque.

Conclusion

This laboratory successfully implemented a FreeRTOS-based embedded system for open-loop DC motor control on the STM32L5 platform. The system fully met all functional requirements, including deterministic task scheduling, high-resolution PWM-based speed control via TIM1 (PE9), GPIO-based direction control (PB10/PB11), and safety-conscious standby management (PE15).

A robust real-time architecture was achieved by strictly separating user interaction from motor actuation. The MotorTask was prioritized to ensure stable hardware control with minimal jitter, while the UITask managed non-critical ADC sampling and speed requests. The use of event-driven synchronization ensured a completely non-blocking execution environment, adhering to best practices for real-time systems design.

All mandatory behaviour was verified: the system reliably initializes to a STOP state with the Red LED active, provides smooth speed variation via the potentiometer, and delivers consistent audible feedback without stalling the scheduler. Overall, this laboratory established a modular and scalable foundation that is well-suited for future enhancements, such as the integration of encoder feedback and closed-loop PID control.